

Public API Guide

CS 7180 — Project 2 Handout

What Is a "Public API"?

In Project 2, your backend must expose **documented RESTful HTTP endpoints** consumable by any client -- not just your own frontend. This is what "Public API with documentation" means.

Key points:

- **"Public" = documented and consumable**, not "open to the internet without authentication." Your API can (and should) require auth -- the point is that it's well-documented enough for any developer to use.
- Your frontend should consume your own API -- no direct database calls from client code.
- Every endpoint must have clear documentation: what it accepts, what it returns, and what errors it produces.
- If your project is a desktop or mobile app without a traditional backend, see Section 6 for equivalences.

REST API Design Essentials

Follow these conventions for a professional-quality API:

- **Resource-based URLs:** Use nouns, not verbs (`/api/v1/tasks`, not `/api/v1/getTasks`)
- **HTTP methods:** GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
- **Proper status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error
- **API versioning:** Prefix routes with `/api/v1/` so you can evolve without breaking clients
- **JSON request/response bodies** with consistent structure
- **Pagination** for list endpoints (`?page=1&limit=20`)
- **Consistent error format** across all endpoints:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Title is required",
    "details": [{ "field": "title", "issue": "must not be empty" }]
  }
}
```

Concrete Example: Task Management App

Here is a minimal but complete API for a task management feature:

Method	Endpoint	Description	Auth
GET	<code>/api/v1/tasks</code>	List tasks (paginated)	Yes
POST	<code>/api/v1/tasks</code>	Create a new task	Yes
GET	<code>/api/v1/tasks/:id</code>	Get a single task	Yes
PATCH	<code>/api/v1/tasks/:id</code>	Update a task	Yes
DELETE	<code>/api/v1/tasks/:id</code>	Delete a task	Yes

Example response for GET `/api/v1/tasks?page=1&limit=2`:

```
{
  "data": [
    { "id": "abc123", "title": "Write API docs", "status": "in_progress" },
    { "id": "def456", "title": "Add pagination", "status": "done" }
  ],
  "pagination": { "page": 1, "limit": 2, "total": 14 }
}
```

Documenting with OpenAPI / Swagger 3.0

OpenAPI is the specification format (a YAML or JSON file describing your API). **Swagger** is the tooling ecosystem that renders it into interactive documentation.

Minimal OpenAPI Snippet

Below is a minimal OpenAPI 3.0 snippet for the GET /api/v1/tasks endpoint:

```
openapi: 3.0.0
info:
  title: Task Manager API
  version: 1.0.0
paths:
  /api/v1/tasks:
    get:
      summary: List all tasks (paginated)
      security:
        - bearerAuth: []
      parameters:
        - name: page
          in: query
          schema: { type: integer, default: 1 }
        - name: limit
          in: query
          schema: { type: integer, default: 20 }
      responses:
        '200':
          description: Paginated task list
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    type: array
                    items:
                      $ref: '#/components/schemas/Task'
                  pagination:
                    $ref: '#/components/schemas/Pagination'
```

Setting Up Swagger UI

Express (recommended for most P2 projects):

```
npm install swagger-ui-express swagger-jsdoc

// server.js or app.js
const swaggerUi = require('swagger-ui-express');
const swaggerJsdoc = require('swagger-jsdoc');

const specs = swaggerJsdoc({
  definition: {
    openapi: '3.0.0',
    info: { title: 'My P2 API', version: '1.0.0' },
  },
  apis: ['./routes/*.js'], // JSDoc comments in route files
});

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));
```

Next.js API Routes:

```
npm install next-swagger-doc swagger-ui-react
```

Create an API route at `/api/doc` that serves the spec, and a page at `/api-docs` that renders Swagger UI.

After setup, visit `http://localhost:3000/api-docs` to see interactive documentation where you can test endpoints directly.

Delivery Checklist

Track A -- Web Apps (REST API)

This is the standard track for most P2 projects.

- Versioned REST endpoints consumed by your frontend (`/api/v1/...`)
- Complete OpenAPI 3.0 spec file (YAML or JSON) checked into your repo
- Swagger UI live at `/api-docs` on your deployed app
- Example request and response documented per endpoint
- Authentication requirements clearly documented
- Error response format documented with examples

Track B -- Local/Desktop Apps (SDK Documentation)

For projects without a traditional backend server (see Section 6).

- All public functions and classes annotated with JSDoc (`@param`, `@returns`, `@example`)
- TypeDoc configuration in repo; generated docs are browsable
- Usage examples for key operations
- Access control and permissions documented
- Error handling patterns documented

Desktop/Mobile App Equivalences

If your P2 is **not** a traditional web app with a separate backend, this section explains how the "Public API with documentation" requirement applies to you.

Scenario A: Mobile/Desktop App WITH a Backend Server

Same requirements as web apps (Track A above). The only difference is that your client is a native app instead of a browser. You still need REST endpoints, an OpenAPI spec, and Swagger UI at `/api-docs`.

Scenario B: Fully Local App (No Separate Server)

Your equivalent of a "public API" is your **module's public interface** -- the exported functions, classes, and types that other developers would use.

What to document:

- Exported public functions and classes with JSDoc annotations
- Function signatures, parameter descriptions, return types
- Usage examples via `@example` tags

Example: TypeDoc-annotated function

```
/**
 * Compress an image file using the specified algorithm.
 *
 * @param inputPath - Absolute path to the source image
 * @param options - Compression settings
```

```

* @param options.quality - Quality level from 0-100 (default: 80)
* @param options.format - Output format: 'jpeg' | 'png' | 'webp'
* @returns Object with output path and compression stats
* @throws {FileNotFoundError} If inputPath does not exist
*
* @example
* const result = await compressImage('/photos/cat.png', {
*   quality: 75,
*   format: 'webp'
* });
* console.log(result.savedBytes); // => 204800
*/
export async function compressImage(
  inputPath: string,
  options: CompressionOptions
): Promise<CompressionResult> { ... }

```

Generate browsable docs:

```

npm install -D typedoc
npx typedoc --entryPoints src/index.ts --out docs

```

Equivalence Table

Web App Requirement	Local App Equivalent
REST endpoints	Exported public functions/classes
OpenAPI spec in repo	TypeDoc config + JSDoc annotations in code
Swagger UI at /api-docs	Generated TypeDoc site (hosted or in repo)
Request/response examples	@example tags in JSDoc
Auth documentation	Access control / permissions documentation

Grading Expectations

The "API quality" category is worth **10 points** under Functionality, and API documentation also contributes to the **15-point Documentation** category. The same quality standards apply whether you follow Track A (REST) or Track B (SDK docs).

Score	API Quality (10 pts)	Documentation Contribution
9-10	All endpoints documented in OpenAPI; Swagger UI deployed; consistent error format; pagination; versioned routes	Comprehensive, accurate, with examples
7-8	Most endpoints documented; Swagger UI works; minor gaps in error format or pagination	Good coverage, a few missing examples
5-6	Partial documentation; some endpoints missing; Swagger UI present but incomplete	Adequate but inconsistent
3-4	Minimal documentation; several undocumented endpoints; no Swagger UI	Sparse, missing key details
0-2	No API documentation; endpoints not RESTful	Missing or unusable

Note: The same point scale applies to both REST API docs (Track A) and SDK/library docs (Track B). What is graded is the **quality and completeness** of your documentation, not the format.